곱셈기 없는 RV32I CPU로 INT8 선형계층 계산하기

EE362 Project 1 · R1.4 공통 flow · 2026-11-17

박기호 (어셈블리 최적화) · 김대현 (CPU 파이프라인)

과제와 평가 규칙

$$Y[j] = BIAS[j] + \Sigma X[i] \cdot W[j][i] \qquad i = 0..7,\ j = 0..3$$

- X·W는 INT8, BIAS·Y는 INT32. 데이터 주소는 case마다 달라 0x300 설정 블록에서 매번 읽는다
- Y 4개를 쓴 뒤 마지막 store로 DONE_ADDR ← DONE_TOKEN. 여기까지가 cycle 수
- 명령: 기본 8개(ADD SUB AND ADDI LW SW BEQ JAL) 필수. 우리는 선택 명령을 쓰지 않았다
- 합성 대상은 TOP(CPU)만. IMEM·DMEM은 TB 쪽, SDC는 주기 숫자만 바꿀 수 있다

점수 (관문 통과 팀끼리 상대평가)

$$S = 12.5 \cdot T_best/T + 6.25 \cdot A_best/A + 6.25 \cdot P_best/P$$

T = 평균 cycle ÷ f

A = CPU cell + IMEM + DMEM 면적

P = CPU + 메모리 동적 + 정적 전력

관문: 24 case 기능, 제어·연결, 재현·개선 근거, 포트·ISA, setup/hold. 하나라도 FAIL이면 PPA 0점

결론: EX 분기 5단 파이프라인 + int8_fastv2, 1.62 ns

요약

1.0210 µs	617.28 MHz	100,342.97	15.4104 mW	16 / 16
실행시간 T	주파수 (1.62 ns)	면적 A (µm²)	전력 P	공개 case PASS

기준본 대비 30.4배 빠르다 = 프로그램 14.75배 × CPU 구조 2.06배

기준본
단일 사이클 3.82 ns
배포 프로그램 (57 word)
8,135.69 cycle
T 31.0783 µs

→

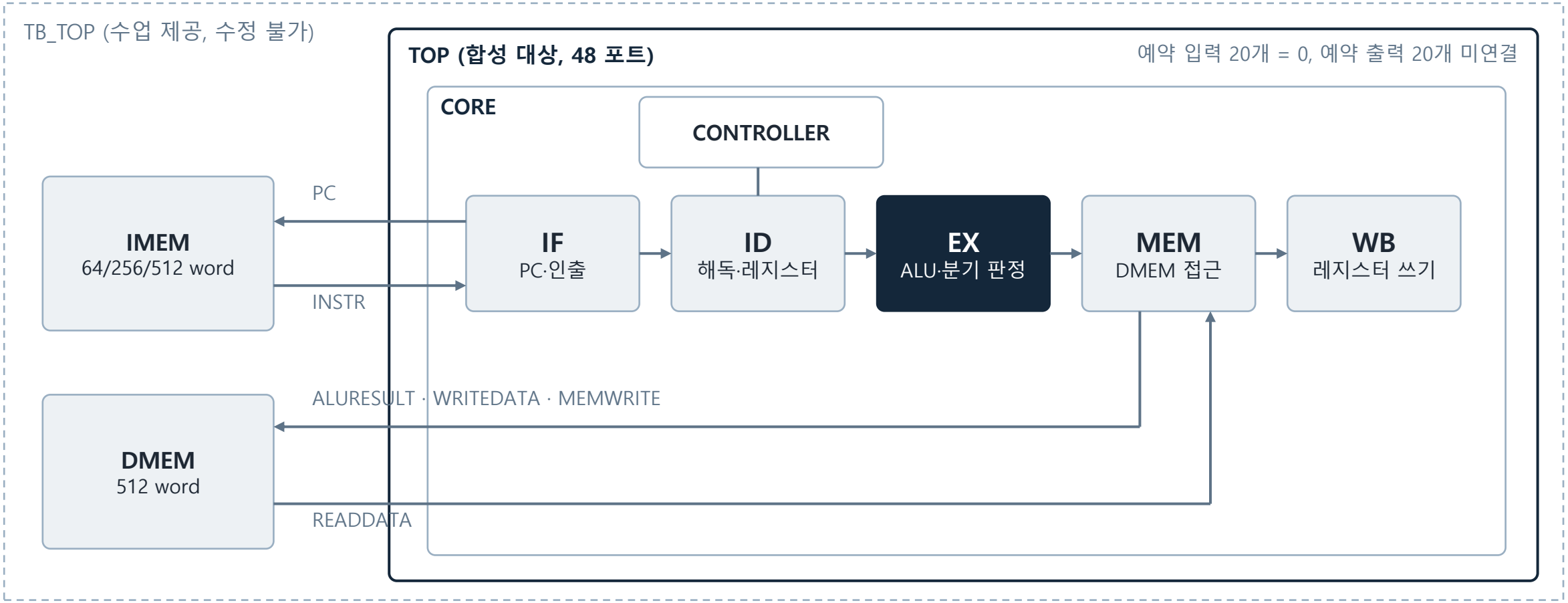
프로그램 개선 ×14.75
단일 사이클 3.82 ns
int8_fastv2 (193 word)
551.50 cycle
T 2.1067 µs

→

CPU 개선 ×2.06
EX 파이프라인 1.62 ns
int8_fastv2
630.25 cycle
T 1.0210 µs

설계: 고정 인터페이스와 5단 파이프라인

피어리뷰 ① 설계 이해



- 분기·JAL은 EX 단에서 판정. taken이면 IF·ID 두 명령을 버린다 (2 cycle)
- 데이터 의존은 MEM·WB → EX 포워딩, lw 직후 사용만 1 cycle stall. 버블은 INSTR 0 → CONTROLLER가 ILLEGAL로 해독해 제어 신호 전부 0

세 CPU의 제어 흐름과 해저드 처리

피어리뷰 ① 설계 이해

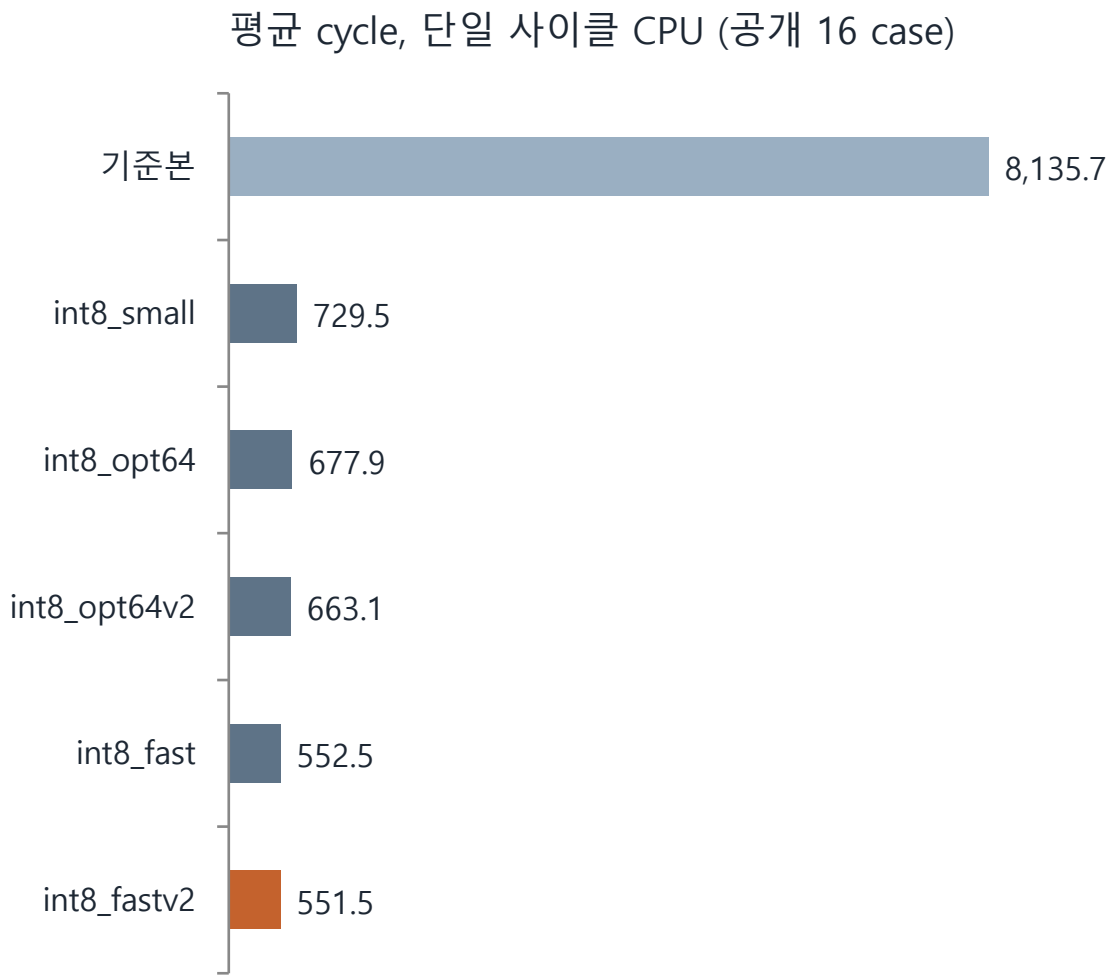
	단일 사이클	파이프라인 ID	파이프라인 EX (선택)
분기·JAL 판정	같은 cycle	ID 단	EX 단
taken 분기·JAL 손실	0	1 cycle	2 cycle
lw 직후 사용	0	1 cycle stall (BEQ는 MEM의 load도 대기)	1 cycle stall
그 밖의 데이터 의존	—	포워딩, 0 cycle	MEM·WB → EX 포워딩, 0 cycle
분기 예측	—	없음 (항상 PC+4)	없음 (항상 PC+4)
불법·미구현 명령	ILLEGAL=1, 상태 변경 없음	같음	같음 (버블도 이 경로)
RESET	동기 active-high, PC=0	같음	같음
최소 통과 주기	3.82 ns	2.36 ns	**1.62 ns**

세 폴더는 rtl/CORE.v만 다르다. TOP·CONTROLLER·TB·메모리·SDC·프로그램 HEX는 같다.

코드 변경 ①: 반복 덧셈 곱을 비트 직렬 곱으로

피어리뷰 ④ 개선 근거

- 기준본: 반복 덧셈 곱 + SCRATCH 저장·재로드
- X[i]의 비트 k가 1이면 네 누산기에 W[j][i]·2^k을 더한다. 비트 7은 부호 비트라 뺀다
- 2^k은 add w,w,w로 매번 2배. 비트 검사 1번(and + beq)을 출력 4행이 같이 쓴다
- 누산기를 BIAS로 시작해 SCRATCH를 쓰지 않는다 → case당 load 50 · store 5
- fast: 루프를 펼쳐 분기를 줄였다. 대가로 IMEM 256 구간
- v2: 같은 알고리즘에서 파이프라인이 버리는 cycle만 줄였다



코드 변경 ②: 주기를 정하는 경로와 RTL 개선

피어리뷰 ④ 개선 근거

가장 느린 경로 (setup 보고서)

단일 3.82 ns
INSTR 입력 → 레지스터 읽기 → ALU 덧셈기 → ALURESULT 출력.
입출력 지연 1.00 ns씩 포함

ID 2.36 ns
MEM 결과 → EX 포워딩 → ALU → ID 비교기 → 분기 판정 → PC.
분기 손실 1 cycle의 대가

EX 1.62 ns
PCE 레지스터 → 분기 목표 덧셈기 → PC 선택

같은 R1.4 flow에서 개선 전 EX 2.30 ns → 1.62 ns, ID 3.00 ns → 2.36 ns.
개선 전 주기는 최소 주기를 찾기 전 값이라, 차이 전부가 RTL 효과는 아니다.

cycle 수를 바꾸지 않은 개선 4가지

1. **BEQ 같음 판정을 ALU와 병렬인 비교기로 (EX)**
   분기 경로에서 뺄셈기 제거
2. **포워딩 select를 ID에서 미리 계산**
   EX의 레지스터 번호 비교기 제거
3. **버블 때 제어 비트만 0**
   flush·stall 신호의 팬아웃 축소
4. **result mux를 MEM 단으로**
   WB 우회 경로 mux 제거, FF 66개 감소

같은 조건에서 기준본과 개선본의 T·A·P

피어리뷰 ③ PPA 비교

단계	CPU · 주기	프로그램	평균 cycle	T (µs)	A (µm²)	P (mW)
기준본	단일 사이클 · 3.82 ns	baseline (57 word)	8,135.69	31.0783	77,533.67	서버 측정 예정
① 프로그램 개선	단일 사이클 · 3.82 ns	int8_fastv2 (193 word)	551.50	2.1067	96,172.47	6.9973
② CPU 개선 (최종)	**EX 파이프라인 · 1.62 ns**	**int8_fastv2**	**630.25**	**1.0210**	**100,342.97**	**15.4104**

T 30.4배 빠름

프로그램 14.75배 × CPU 구조 2.06배

A +29.4%

IMEM 256 구간 +18,638.80, 파이프라인 레지스터 등 CPU +4,170.50 µm²

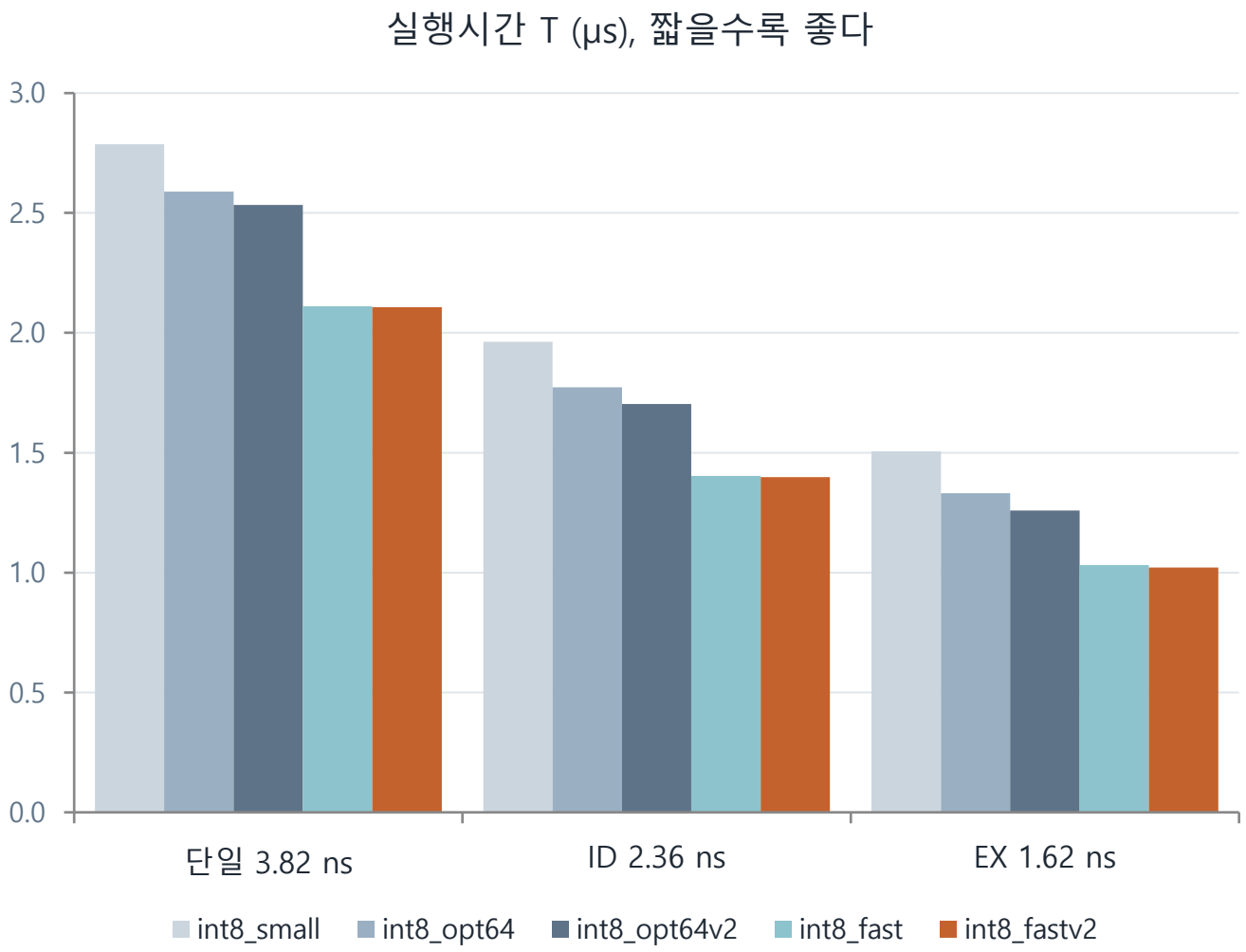
P 2.20배 (① 대비)

주파수 2.36배라 CPU 전력도 2.70배

기준본 A는 단일 사이클 3.82 ns 합성 결과(프로그램과 무관) + IMEM 64 + DMEM. 기준본 T는 공개 16 case 평균 cycle × 3.82 ns.

PPA 15개 조합 (공식 flow, 공개 16 case)

피어리뷰 ③ PPA 비교



int8_fastv2	단일	ID	EX
주기 (ns)	3.82	2.36	**1.62**
평균 cycle	551.50	592.38	630.25
T (µs)	2.1067	1.3980	**1.0210**
A_CPU (µm²)	18,152.49	22,837.38	22,322.99
A 전체 (µm²)	96,172.47	100,857.36	100,342.97
P_CPU (mW)	3.2658	6.4338	8.8303
P 전체 (mW)	6.9973	11.4706	15.4104

A는 프로그램이 64 word 이하면 IMEM 64 구간이라 18,638.80 µm² 작다.

트레이드오프: 무엇을 얻고 무엇을 냈나

피어리뷰 ⑥ 한계·trade-off

1.46×

ID 판 주기 ÷ EX 판 주기

ID는 cycle이 6.0% 적지만 주기가 45.7% 길어 T가 36.9% 길다. 원인은 ID의 'ALU → 비교기 → PC' 경로

−18.9%

fastv2의 T (opt64v2 대비, EX)

대신 A +22.8%, P +30.0%. T 배점이 A·P의 2배라 fastv2. 표 안 점수 차는 0.62로 작다

2.20×

EX의 P ÷ 단일 사이클의 P

주파수만큼 스위칭이 는다. 한 번 실행 에너지 P×T는 14.74 / 16.04 / 15.73 nJ로 비슷하다 (채점 항목 아님)

77.8%

전체 면적 중 메모리 몫 (EX + fastv2)

CPU 구조를 바꿔도 A는 5% 안에서 움직인다. A를 줄이는 길은 64 word 안에 넣는 것

기능 증거: Y·DONE과 검사 범위

서버 VCS · EX 1.62 ns · int8_fastv2

```
PASS CPU demo 624 cycles
PASS CPU large_bias 624 cycles
PASS CPU max_positive 682 cycles
PASS CPU mixed_sign 622 cycles
PASS CPU negative_extreme 586 cycles
PASS CPU random_00 634 cycles
   ...  random_01 ~ random_09
PASS CPU zero_weights 624 cycles
TOTAL CPU PASS 16/16.
```

case마다 load 50 · store 5 (Y 4 + DONE 1)

어디서	무엇을	결과
서버 공식 flow	15개 조합 × 공개 16 case	모두 16/16, timing PASS
로컬 ModelSim	store 순서·주소·데이터·최종 DMEM을 ISA 모델과 비교. 실행 중 RESET, 해저드 시험, 무작위 프로그램 150개	CPU마다 219/219
로컬 ModelSim	v2 프로그램 × 세 CPU × 16 case	모두 PASS
계약 검사	TOP 48 포트, CPU RTL 3개, 공통 SDC	통과

정답 조건: Y 4 word 일치 → 마지막 store가 DONE_ADDR ← DONE_TOKEN, 입력·설정 영역 훼손 없음, timeout 없음

재현 절차: 파일·명령·조건

피어리뷰 ⑤ 재현 방법

1 R1.4 패키지에 R1.7 업데이트 적용 (R1.5·R1.6 포함)

```
unzip -o ../Project1_R1p7_Update_20261112.zip
```

2 어셈블 (HEX·LST 재생성)

```
python3.12 tools/rv32i_assembler.py programs/int8_fastv2.s -o
programs/int8_fastv2.hex --listing programs/int8_fastv2.lst
```

3 주기 설정 (이 숫자만 바꾼다)

```
synth/common.sdc:  create_clock -name CPU_CLK -period 1.62 ...
```

4 기능 + activity (VCS)

```
bash run_sim.sh --program programs/int8_fastv2.hex
```

5 합성 · STA · 전력 · DDC (DC)

```
bash run_ppa.sh   →  results/ppa_summary.json, results/TOP_mapped.ddc,
reports/cpu_hierarchy.rpt
```

조건 SAED32 RVT SS 0.95 V 125 °C · DC compile (compile_ultra 아님), R1.6·R1.7 Tcl은 CPU 계층 유지 · 공통 SDC · RESET=0 정상 모드 STA · 공개 16 case

R1.7 최종 실행 fingerprint e486b618…a2260 · DDC 416 KB · 측정 기록 measurements/R1p7_20261114

한계와 예상 질문

피어리뷰 ⑥ 한계·trade-off

한계

- 숫자는 공개 16 case. 공식 성적은 숨김 8 case를 더한 24 case로 따로 나온다
- setup 여유가 EX 2.6 ps. RTL을 한 줄이라도 고치면 주기를 다시 찾는다
- 통과 주기가 단조롭지 않다 (EX 1.64 실패, 1.62 통과). 합성이 주기마다 다른 회로를 만든다
- 전력은 배치·배선 전 추정, 메모리 전력은 수업용 산식
- 선택 명령(BNE 등)은 쓰지 않았다. 루프 분기를 더 줄일 여지가 있다
- R1.6·R1.7 Tcl(CPU 계층 유지)로 다시 돌려도 T·A·P·slack이 R1.4 Tcl 값과 같았다. 제출 DDC는 R1.7 실행의 것

Q. 왜 ID가 아니라 EX인가?
A. cycle은 ID가 6% 적지만 T는 EX가 36.9% 짧다 (주기 경로)

Q. 왜 IMEM 256 구간 프로그램인가?
A. T −18.9%가 A +22.8%, P +30.0%보다 크다 (배점 2:1:1)

Q. MUL을 쓰면 더 빠르지 않나?
A. M 확장은 금지. 비트 직렬 곱이 곱셈기를 대신한다

Q. 1.62 ns를 믿을 수 있나?
A. 같은 RTL·flow면 결정적이다. R1.4 두 번, R1.6·R1.7 각 한 번 모두 slack +2.6 ps

정리

- EX 분기 5단 파이프라인 + int8_fastv2, 1.62 ns: T 1.0210 µs, 공개 16/16 PASS
- 기준본 대비 30.4배 = 비트 직렬 곱(14.75배) × 파이프라인과 경로 단축(2.06배)
- 대가: 면적 +29.4%, 전력 증가. T 가중치 2배라 이 선택이 점수에 유리
- R1.7 공통 flow 재실행에서 같은 수치, 계층 DDC(CORE·CONTROLLER) 생성

부록 A. PPA 15개 조합 전체

부록

CPU	프로그램	주기	평균 cycle	T (μs)	A (μm²)	P (mW)	표 안 점수
단일 사이클	int8_small	3.82	729.50	2.7867	77,533.67	5.2745	17.08
단일 사이클	int8_opt64	3.82	677.88	2.5895	77,533.67	5.3181	17.38
단일 사이클	int8_opt64v2	3.82	663.13	2.5331	77,533.67	5.3109	17.50
단일 사이클	int8_fast	3.82	552.50	2.1105	96,172.47	6.9959	15.80
단일 사이클	int8_fastv2	3.82	551.50	2.1067	96,172.47	6.9973	15.81
파이프라인 ID	int8_small	2.36	831.38	1.9620	82,218.56	8.8657	16.12
파이프라인 ID	int8_opt64	2.36	751.13	1.7727	82,218.56	8.9327	16.78
파이프라인 ID	int8_opt64v2	2.36	721.63	1.7030	82,218.56	8.9382	17.08
파이프라인 ID	int8_fast	2.36	594.38	1.4027	100,857.36	11.5112	16.77
파이프라인 ID	int8_fastv2	2.36	592.38	1.3980	100,857.36	11.4706	16.81
파이프라인 EX	int8_small	1.62	929.25	1.5054	81,704.17	11.8007	17.20
파이프라인 EX	int8_opt64	1.62	821.38	1.3306	81,704.17	11.8878	18.30
파이프라인 EX	int8_opt64v2	1.62	777.13	1.2589	81,704.17	11.8581	18.85
파이프라인 EX	int8_fast	1.62	636.25	1.0307	100,342.97	15.5594	19.33
파이프라인 EX	**int8_fastv2**	**1.62**	**630.25**	**1.0210**	**100,342.97**	**15.4104**	**19.47**

표 안 점수 = 12.5·T_best/T + 6.25·A_best/A + 6.25·P_best/P (이 15행 기준). 실제 점수는 다른 팀 최솟값이 기준이라 다르다.

부록 B. 주기 탐색 (공식과 같은 compile, 0.02 ns 간격)

부록

단일 사이클

주기 (ns)	setup slack (ns)	판정
3.90	0.0005	통과
3.88	0.0020	통과
3.86	−0.0020	실패
3.84	−0.0037	실패
3.82	**0.0008**	통과
3.80	−0.0000	실패

파이프라인 ID

주기 (ns)	setup slack (ns)	판정
2.40	0.0016	통과
2.38	0.0008	통과
2.36	**0.0003**	통과
2.34	−0.0006	실패
2.32	−0.0055	실패
2.30	−0.0019	실패

파이프라인 EX

주기 (ns)	setup slack (ns)	판정
1.70	0.0031	통과
1.68	0.0007	통과
1.66	0.0002	통과
1.64	−0.0014	실패
1.62	**0.0026**	통과
1.60	−0.0000	실패

- 1차 탐색(0.05~0.30 ns 간격)의 최소 통과 주기 3.90 / 2.40 / 1.70 ns 아래를 다시 찾았다. 음영이 고른 주기
- hold slack은 모든 주기에서 0.1030 ns 이상. 고른 주기의 공식 flow slack은 탐색 값과 소수 넷째 자리까지 같았다
- 주기는 소수 둘째 자리까지만 쓴다 (TB 시간 단위 1 fs, cycle × 주기 = 실행 구간 검사)

부록 C. 프로그램별 크기와 cycle

부록

프로그램	word	IMEM	단일	ID	EX	v2로 줄어든 cycle (EX)
기준본 baseline	57	64	8,135.69	10,186.38	12,170.06	
int8_small	48	64	729.50	831.38	929.25	
int8_opt64	58	64	677.88	751.13	821.38	
int8_opt64v2	63	64	663.13	721.63	777.13	−5.4% (opt64 대비)
int8_fast	192	256	552.50	594.38	636.25	
int8_fastv2	193	256	551.50	592.38	630.25	−0.9% (fast 대비)

- opt64v2: 다음 비트가 0이면 JAL을 건너뛰고, 음수 X의 부호 처리 뒤 JAL을 없앴다 (63 word, IMEM 64 유지)
- fastv2: lw 직후 사용 stall을 없애고 설정 명령 1개를 줄였다 (193 word)
- 두 v2 모두 분기 뒤에 잘못 인출되는 lw/sw가 메모리 접근으로 세어지지 않게 채움 명령을 넣었다
- 기준본 cycle은 로컬 시뮬레이션 값 (서버 기준본 측정은 예정)

부록 D. 제출물 구성 (R1.5~R1.7 지침)

부록

```
TEAM_ID/
  README.txt
  rtl/TOP.v  CORE.v  CONTROLLER.v
  asm/int8_fastv2.s  .hex  .lst
  synth/common.sdc (1.62)  rtl.f
  tools/  (Python 3.6용 공식 도구 3개)
  run/reproduce.sh
  results/ppa_summary.json  ppa_status.json
          simulation_manifest.json
          TOP_mapped.ddc   ← R1.6 필수
          TOP_mapped.v  TOP_normal_mode.sdc
  reports/ setup hold cpu_area
          cpu_hierarchy  case별 power
  logs/    compile  case 16개  dc_ppa
  presentation/project1.pptx  .pdf
```

- 발표 슬라이드 1부 (PPTX 또는 PDF). 서술형 보고서는 내지 않는다
- RTL, 어셈블리 원본 .s, 재생성한 .hex·.lst
- 기능 로그, STA·CPU area·CPU power 도구 출력, 선언 주파수 617.28 MHz
- 실행 cycle과 load/store 횟수, IMEM 용량 (193 word → 256 구간)
- 사용한 선택 RV32I 명령: 없음 (기본 8개만). 그래서 단독 기능 시험 대상 없음
- R1.6: 계층 DDC results/TOP_mapped.ddc 필수. CORE·CONTROLLER 계층을 reports/cpu_hierarchy.rpt로 확인하고, 같은 실행의 DDC와 측정 결과를 낸다
- 재현 절차 (슬라이드 12와 같은 명령)

부록 E. 수업 명령만으로 서버에서 재현

피어리뷰 ⑤ 재현 방법

1 컴파일 (VCS)

```
vcs_queue -full64 -timescale=1ns/1fs -top TB_TOP -f synth/rtl.f \
  sim/mem/IMEM.v sim/mem/DMEM.v sim/MEMORY_POWER.v sim/TB_TOP.v -o build/simv -l logs/compile.log
```

2 16 case 실행

```
for f in cases/*.json; do c=$(basename $f .json)
  ./build/simv +PROGRAM=programs/int8_fastv2.hex +DATA=cases/${c}_dmem.hex +EXPECTED=cases/${c}_expected.hex \
    +F_MHZ=617.283950617 +VCD=build/$c.raw.vcd -l logs/$c.log; done
```

3 실행 구간 → SAIF

```
trim_vcd.py(배포 도구)로 로그의 ACTIVE_WINDOW 구간만 남김
→ vcd2saif -input build/$c.active.vcd -output build/$c.saif
```

4 합성·STA (DC)

```
dc_shell_queue → source synth/library_setup.tcl · saif_map -start · analyze · elaborate TOP · link
→ source synth/common.sdc · set_ungroup … false · compile · write_file -format ddc -hierarchy · report_hierarchy
→ set_case_analysis 0 [get_ports RESET] · report_area · report_timing -delay_type max / min
```

5 전력 (DC)

```
foreach c $P1_CASES { reset_switching_activity
  read_saif -input build/$c.saif -instance_name TB_TOP/dut -auto_map_names; report_power > reports/$c/power.rpt }
```

6 T·A·P 계산

```
case별 cycle·load·store(logs)와 CPU 전력(power.rpt)을 awk로 cycle 가중 평균 + 메모리 산식
→ T 1.021005 µs · A 100,342.97 µm² · P 15.4104 mW (공식 결과와 같음)
```

전체 명령: 팀원용 HTML "서버에서 재현" · 공식 2줄 판은 슬라이드 12